

Session 19

1.Download a sample Terms & Conditions document from any popular app (like Zomato, Flipkart, or Instagram), and perform a static review to identify at least 5 unclear or ambiguous statements that could confuse users.

1. Static Review of a Terms & Conditions Document

Document selected: Zomato Terms & Conditions

Testing type: Static Testing

Method: Review the document without running the application.

No .	Unclear/Ambiguous Statement	Why It May Confuse Users	Suggested Improvement
1	"Zomato may modify the services from time to time."	Users may not know what type of changes can be made.	Clearly mention examples of possible changes.
2	"Additional charges may apply."	It does not clearly explain which charges may be added.	Mention the types of charges and when they apply.
3	"Offers are subject to certain conditions."	Users may not know what the conditions are.	Provide a clear list of offer conditions.
4	"Zomato may suspend or terminate an account."	The reason for suspension is not always clear from the statement.	Explain common reasons and the user notification process.
5	"Delivery times are estimates only."	Users may not know how much the actual delivery time can vary.	Clearly explain that delivery time can change because of traffic, weather, restaurant delays, etc.

Conclusion:

Static review helps identify unclear wording, missing information, and possible misunderstandings before users are affected.

2.Create a comparison table listing 3 key differences between static testing and dynamic testing, using real-life examples from apps you use daily (such as a privacy policy review vs. actually filling a signup form).

Static Testing vs Dynamic Testing

Point	Static Testing	Dynamic Testing
Meaning	Testing without executing the application.	Testing by actually running the application.
Example	Reviewing an app's Privacy Policy for unclear statements.	Opening the signup page and actually entering user details.
Main Purpose	Find errors in documents, requirements, design, or code early.	Find functional errors while the application is running.
Example from daily apps	Reviewing Zomato's Terms & Conditions.	Placing a food order on Zomato and checking whether payment works.

3 Key Differences

1. **Execution:** Static testing does not execute the software, while dynamic testing executes it.
2. **Focus:** Static testing checks documents, requirements, designs, and code; dynamic testing checks actual application behavior.
3. **Timing:** Static testing can find problems early, while dynamic testing is generally performed when an executable version is available.

3.Imagine you are part of a code walkthrough for a new feature in a food delivery app (like Swiggy) — write 5 questions or points you would raise while reviewing the code for the 'Apply Coupon' function.

Code Walkthrough – Apply Coupon Function

Feature: Apply Coupon in a food delivery app like Swiggy.

During a code walkthrough, I would raise these 5 questions:

1. Coupon Validity

Question:

Does the code check whether the coupon has expired?

Reason:

An expired coupon should not be accepted.

2. Minimum Order Value

Question:

Does the code check the minimum cart value required for the coupon?

Reason:

For example, a ₹500 minimum-order coupon should not work for a ₹300 order.

3. User Eligibility

Question:

Does the code check whether the user is eligible for the coupon?

Reason:

Some coupons may be available only for new users or selected users.

4. Multiple Coupon Usage

Question:

Does the code prevent a user from applying the same coupon more than the allowed number of times?

Reason:

This prevents misuse of coupons.

5. Correct Discount Calculation

Question:

Is the discount calculated correctly for both percentage and fixed-amount coupons?

Reason:

For example, a 20% discount and a ₹100 discount should be calculated differently.

Conclusion:

The walkthrough helps identify logic errors and missing validations before the code reaches testing or production.

4. Conduct a mock inspection of a sample bug report from any open-source project (find one on GitHub or use a template online), and list 3 improvements you would suggest to make the report clearer and more useful for developers.
Hint: Look for missing steps, unclear expected results, or vague descriptions.

Mock Inspection of a Bug Report

Sample Bug: Apply Coupon feature does not apply a valid coupon.

Original/Weak Bug Report

Title: Coupon not working

Description: I tried applying a coupon but it didn't work. Please fix it.

This report is too vague because developers do not have enough information to reproduce the problem.

3 Improvements

No.	Improvement	Example
1	Add clear reproduction steps	1. Login → 2. Add items worth ₹600 → 3. Open Coupon section → 4. Enter SAVE100 → 5. Tap Apply
2	Clearly mention expected result	"The ₹100 discount should be applied to the order."
3	Clearly mention actual result	"The coupon shows an error message: 'Unable to apply coupon' even though the coupon is valid."

Improved Bug Report

Bug Title: Valid **SAVE100** coupon is not applied to an order above ₹500

Steps to Reproduce:

1. Login to the application.
2. Add food items worth ₹600 to the cart.
3. Open the Coupon section.
4. Enter **SAVE100**.

5. Click **Apply**.

Expected Result:

The ₹100 discount should be applied to the order.

Actual Result:

The coupon is not applied and an error message is displayed.

Conclusion:

A clear bug report should contain **steps to reproduce, expected result, actual result, and enough information for the developer to reproduce the issue.**

Short Final Conclusion

Static testing is useful because it can find problems **before the software is executed.**

Reviews, walkthroughs, inspections, and document checking can reduce defects early and improve the quality of the final application.